

RXJS ДЛЯ ANGULAR-РАЗРАБОТЧИКОВ

RxJS

Реактивное программирование для Angular-разработчиков

Ментальная модель потоков данных для решения реальных UI-задач



stream of values over time



Проблема и интуиция

Почему привычных инструментов недостаточно и что такое поток

Почему callback и Promise недостаточны

UI — это множество событий во времени, а не один результат

Callbacks

Вложенность, сложность отмены,
невозможно комбинировать источники

Promise

Один результат, без отмены, нельзя передать
несколько значений

addEventListener

Ручная очистка, нет трансформации, хаос
при множестве источников

⚡ Promise — одно значение. UI — много значений во времени.

Что такое RxJS

Библиотека для работы с потоками значений во времени — единый подход к событиям, HTTP и данным



Поток значений, распределённых во времени — клики, ввод, HTTP, таймеры



Операторы для трансформации, фильтрации и комбинирования потоков



Единый API для всех асинхронных источников в приложении



RxJS — это Lodash для событий: те же `map` и `filter`, но для значений во времени.

Promise vs Observable

Promise — одно значение сразу. Observable — поток значений, который можно отменить

Promise

- ✗ Одно значение
- ✗ Не отменяется
- ✗ Выполняется сразу
- ✓ Простота

Observable

- ✓ Много значений
- ✓ Отменяемый
- ✓ Ленивый (lazy)
- ✓ Композируемый

⚡ Promise стартует сразу. Observable стартует только при `subscribe()`.

Где живёт Observable

Observable — единая обёртка для всех источников данных в Angular

HTTP

`HttpClient.get()` возвращает Observable

Events

`fromEvent(element, 'click')` — клики, ввод

Forms

`FormControl.valueChanges` — поток значений

Router

`ActivatedRoute.params` — поток параметров URL

WebSocket

Поток сообщений в реальном времени

Timer

`interval(1000)` — периодические значения

 В Angular почти всё асинхронное — это Observable.

Push и Pull

Pull — вы запрашиваете. Push — данные приходят сами.

Pull

Вы вызываете функцию → получаете результат

```
const answer = service.getData();
```

Push

Источник сам отправляет значения, когда они появляются

```
stream$.subscribe(v => console.log(v));
```

 Observable — это Push-модель: данные приходят сами, вы только подписываетесь.



Базовые элементы

Observable, Observer, Subscription, pipe, operators

Observable — ленивый поток

Observable — это рецепт. Он ничего не делает, пока вы не подпишетесь.

1 Создание: `const stream$ = of(1, 2, 3);` — ничего не произошло

2 Подписка: `stream$.subscribe(v => log(v));` — значения пошли

```
import { of } from 'rxjs';

const stream$ = of(1, 2, 3);
// Ничего не происходит

stream$.subscribe(v => console.log(v));

// 1, 2, 3
```

⚡ Без `subscribe()` Observable — просто описание, не действие.

Жизненный цикл: next, error, complete

У каждого Observable три типа уведомлений

next(value)

Новое значение в потоке. Может вызываться 0+ раз.

error(err)

Ошибка. Поток завершается. Дальше — ничего.

complete()

Поток завершён успешно. Дальше — ничего.

```
stream$.subscribe({  
  
  next:    v => console.log(v),  
  error:   e => console.error(e),  
  complete: () => console.log('done')  
});
```

⚡ Контракт: $\text{next}^* \rightarrow (\text{error} \mid \text{complete})?$ — после error или complete новых значений не будет.

Subscription и очистка

Подписка — это ресурс. Незакрытая подписка = утечка памяти.

- `subscribe()` → Начинается приём значений
- `unsubscribe()` → Приём значений останавливается, ресурсы освобождаются
- забыли отписаться → Утечка памяти, зомби-коллбэки

```
const sub = interval(1000).subscribe(v => log(v));  
// ...позже...  
  
sub.unsubscribe(); // остановить поток
```

ANGULAR В Angular используйте `takeUntil(destroy$)` или `AsyncPipe` — он отписывается сам.

⚡ `interval(1000)` без отписки будет работать вечно — даже после уничтожения компонента.

Что теперь умеем

✓ Объяснить разницу между Promise и Observable

✓ Создать Observable, подписаться и отписаться

✓ Понять контракт next → error | complete

✓ Знать, почему незакрытая подписка — это утечка



Операторы создания

of, from, interval, defer, throwError

of()

Создаёт Observable из готовых значений. Каждое значение — next, затем complete.

input
↓
↓
output

```
import { of } from 'rxjs';  
  
of(10, 20, 30).subscribe({  
  
  next: v => console.log(v),  
  complete: () => console.log('done')  
});  
  
// 10, 20, 30, done
```

⚡ of — самый простой способ создать Observable из значений.

from()

Превращает массив, Promise или iterable в Observable — по одному значению за раз.

input
↓
↓
output

```
import { from } from 'rxjs';

from(['a', 'b', 'c']).subscribe(v => log(v));

// 'a', 'b', 'c' — по одному

from(fetch('/api/data')).subscribe(res => log(res));
// Promise → Observable
```

⚡ from разворачивает массив в поток значений. of — передаёт массив целиком.

of([1,2,3]) vs from([1,2,3])

of даёт массив одним значением. from разворачивает массив в отдельные значения.

of([1, 2, 3])

[1, 2, 3]

next([1,2,3]) → complete()

from([1, 2, 3])

1

2

3

next(1) → next(2) → next(3) → complete()

```
of([1, 2, 3])  
  .subscribe(v => console.log(v));
```

// [1, 2, 3] ← одно значение-массив

```
from([1, 2, 3])  
  .subscribe(v => console.log(v));
```

// 1, 2, 3 ← три значения

⚡ of хранит структуру. from разворачивает.

from(promise)

Promise превращается в Observable: один next, затем complete. Ошибка Promise → error.

Promise
↓ from()
Observable →

```
import { from } from 'rxjs';
import { HttpClient } from '@angular/common/http';

from(fetch('/api/users'))

  .subscribe({
    next: res => log(res),
    error: err => log(err)
  });
```

ANGULAR Когда нужен Observable из стороннего Promise (например, Web API).

⚡ Promise resolve → next + complete. Promise reject → error.

defer()

Создаёт Observable заново при каждой подписке. Фабрика, которая вызывается на каждый subscribe.

subscribe #1 → factory() вызывается → новый Observable

subscribe #2 → factory() вызывается снова → новый Observable

```
import { defer, of } from 'rxjs';

const time$ = defer(() => of(Date.now()));

time$.subscribe(v => log(v)); // 1698765432000

time$.subscribe(v => log(v)); // 1698765432100 (другое!)
```

ANGULAR HTTP-запрос, который нужно повторять с актуальными параметрами при каждой подписке.

⚡ of(1) — одно значение для всех. defer — новый Observable на каждый subscribe.

throwError()

Создаёт `Observable`, который сразу отправляет `error`. Полезен в `catchError` для повторного выброса.

Подписка → сразу `error(err)`

```
import { throwError } from 'rxjs';

throwError(() => 'Boom').subscribe({

  error: e => console.log(e) // 'Boom'
});
```

⚡ `throwError` нужен в `catchError`: вернуть ошибку дальше, а не «проглотить» её.

interval() + take()

`interval(period)` — бесконечный поток с задержкой. `take(N)` — берёт N значений и завершает.

input
↓
↓
output

```
import { interval, take } from 'rxjs';

interval(1000)           // 0, 1, 2, 3, ... бесконечно
  .pipe(take(3))         // взять только 3

  .subscribe(v => log(v));
// 0, 1, 2, complete
```

⚡ `take(N)` — простой способ завершить бесконечный поток.

Что теперь умеем

✓ Создать Observable из значений, массива, Promise

✓ Использовать defer для ленивого создания на каждую подписку

✓ Понять разницу of и from

✓ Прервать бесконечный поток через take()

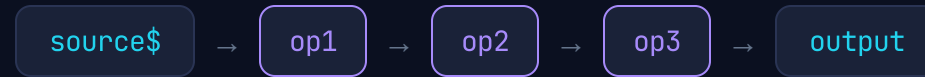


pipe и операторы

Трансформация, фильтрация и побочные эффекты

pipe()

pipe — это конвейер. Значения проходят через операторы слева направо.



```
source$.pipe(  
  op1,  
  op2,  
  op3  
)  
.subscribe(result);
```

 Операторы в pipe выполняются последовательно: вывод op1 → вход op2.

map()

Преобразует каждое значение. Как `Array.map`, но для потока.

input
↓
↓
output

```
import { of, map } from 'rxjs';

of(1, 2, 3).pipe(
  map(x => x * 10)
).subscribe(v => log(v));
// 10, 20, 30
```

ANGULAR Из ответа API → модель для шаблона (ViewModel).

⚡ `map` — чистая трансформация: вход → выход, без побочных эффектов.

filter()

Пропускает только значения, удовлетворяющие условию.

input
↓
↓
output

```
import { of, filter } from 'rxjs';  
  
of(1, 2, 3, 4, 5).pipe(  
  filter(x => x > 2)  
)  
.subscribe(v => log(v));  
// 3, 4, 5
```

ANGULAR Фильтрация элементов списка по условию из valueChanges.

⚡ filter пропускает или отбрасывает. Он не меняет значение.

map + filter в pipe

Операторы комбинируются в цепочку: вывод одного — вход следующего.

```
input
↓ filter(x => x > 2)
  filtered
↓ map(x => x * 10)
  output
```

```
of(1, 2, 3, 4, 5).pipe(
  filter(x => x > 2),

  map(x => x * 10)

).subscribe(v => log(v));
// 30, 40, 50
```

⚡ Порядок в pipe важен: filter → map и map → filter дают разные результаты.

tap()

Выполняет побочное действие, не меняя поток. Для логов, отладки, загрузки.

input
↓ tap(v => log(v))
output

значения не изменились, но лог сработал

```
of(1, 2, 3).pipe(  
  tap(v => console.log('got:', v)),  
  
  map(v => v * 10)  
).subscribe();
```

ANGULAR Логирование значений потока для отладки без изменения результата.

⚡ tap не трансформирует. Если нужно изменить — используйте map.

startWith()

Добавляет начальное значение перед потоком. Идеально для состояния загрузки.

input
↓
↓
output

```
import { of, startWith } from 'rxjs';
```

```
of('Loaded').pipe(
```

```
  startWith('Loading')
```

```
).subscribe(state => log(state));
```

```
// 'Loading', 'Loaded'
```

ANGULAR Показать 'Loading...' до получения данных от HttpClient.

⚡ startWith — первое, что увидит подписчик. Отлично для UI-состояний.

distinctUntilChanged()

Пропускает значение только если оно отличается от предыдущего.

input
↓
↓
output

```
import { from, distinctUntilChanged } from 'rxjs';

from(['a', 'a', 'b', 'b', 'a']).pipe(
  distinctUntilChanged()
).subscribe(v => log(v));
// 'a', 'b', 'a' – без дублей подряд
```

ANGULAR Фильтр form.valueChanges: не запускать логику, если значение не изменилось.

⚡ distinctUntilChanged сравнивает с предыдущим. Для объектов нужен компаратор.

debounceTime()

Ждёт паузу в потоке. Пропускает значение только если N мс ничего не пришло.

input
↓
↓
output

```
import { fromEvent, debounceTime } from 'rxjs';  
  
fromEvent(input, 'input').pipe(  
  
  debounceTime(300)  
  
).subscribe(term => search(term));  
// запрос только после паузы 300мс
```

ANGULAR Live search: не отправлять запрос на каждый символ, ждать окончания ввода.

⚡ debounceTime = «подожди, пока пользователь перестанет печатать».

Что теперь умеем

✓ Комбинировать операторы через `pipe()`

✓ Фильтровать поток через `filter`

✓ Создавать UI-состояния через `startWith`

✓ Ждать паузу ввода через `debounceTime`

✓ Трансформировать значения через `map`

✓ Добавлять побочные эффекты через `tap`

✓ Убирать дубли через `distinctUntilChanged`

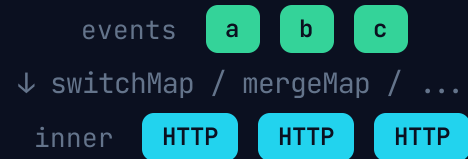


Higher-order mapping

`switchMap`, `mergeMap`, `concatMap`, `exhaustMap`

Что такое higher-order mapping

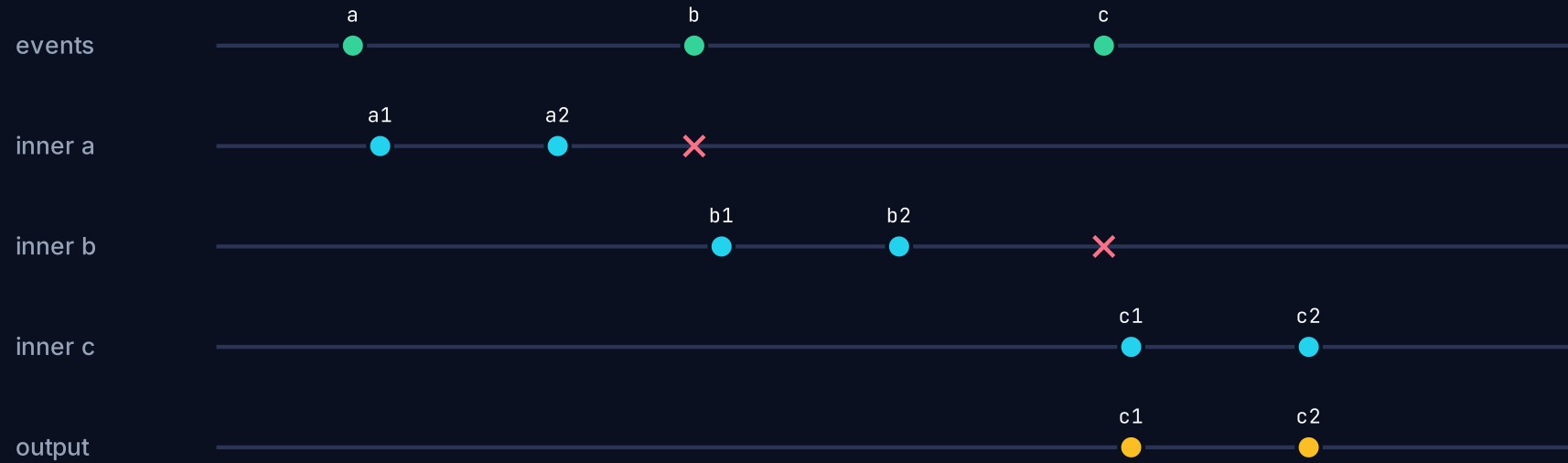
Когда значение потока — тоже `Observable`. Оператор решает, как управлять внутренними потоками.



Каждое событие запускает внутренний `Observable` (например, HTTP-запрос). Как управлять несколькими?

⚡ Все четыре оператора — это `map` + стратегия управления внутренними `Observable`.

switchMap — отменяет предыдущий



Новое событие → предыдущий внутренний поток отменён
Результат: только последний внутренний поток доходит до конца

switchMap — когда использовать

Отменяет предыдущую внутреннюю операцию при новом событии.

ПОВЕДЕНИЕ

Новое значение → отписывается от предыдущего
внутреннего → подписывается на новый

ИДЕАЛЬНО ДЛЯ

Live search, отмена устаревших HTTP-запросов

ОПАСНО ДЛЯ

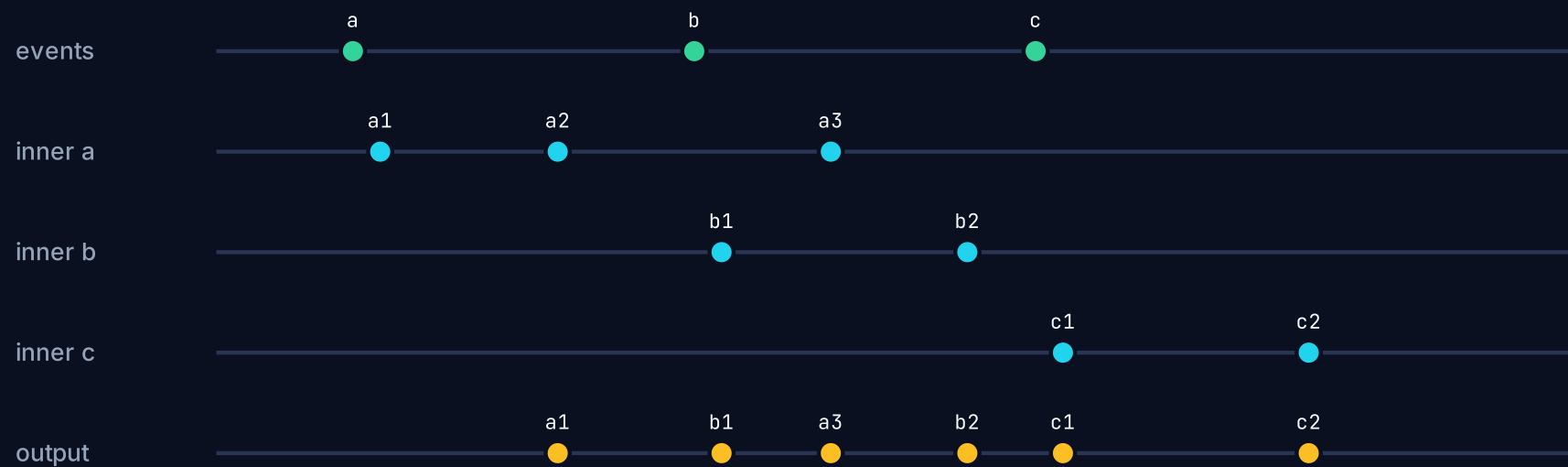
Сохранение формы — запрос отменится до завершения

```
searchControl.valueChanges.pipe(  
  debounceTime(300),  
  
  switchMap(term => http.get('/api/search?q=' + term))  
  
).subscribe(results => show(results));
```

ANGULAR Поле поиска: новый ввод отменяет предыдущий запрос — пользователь видит только актуальные результаты.

⚡ switchMap = «забудь прошлое, делай новое».

mergeMap — выполняет параллельно



Все внутренние потоки выполняются одновременно
Результаты приходят в порядке завершения

mergeMap — когда использовать

Все внутренние **Observable** выполняются параллельно, результаты приходят по мере готовности.

ПОВЕДЕНИЕ

Каждое событие запускает новый внутренний поток. Все живут одновременно.

ИДЕАЛЬНО ДЛЯ

Логирование, фоновые задачи, независимые запросы

ОПАСНО ДЛЯ

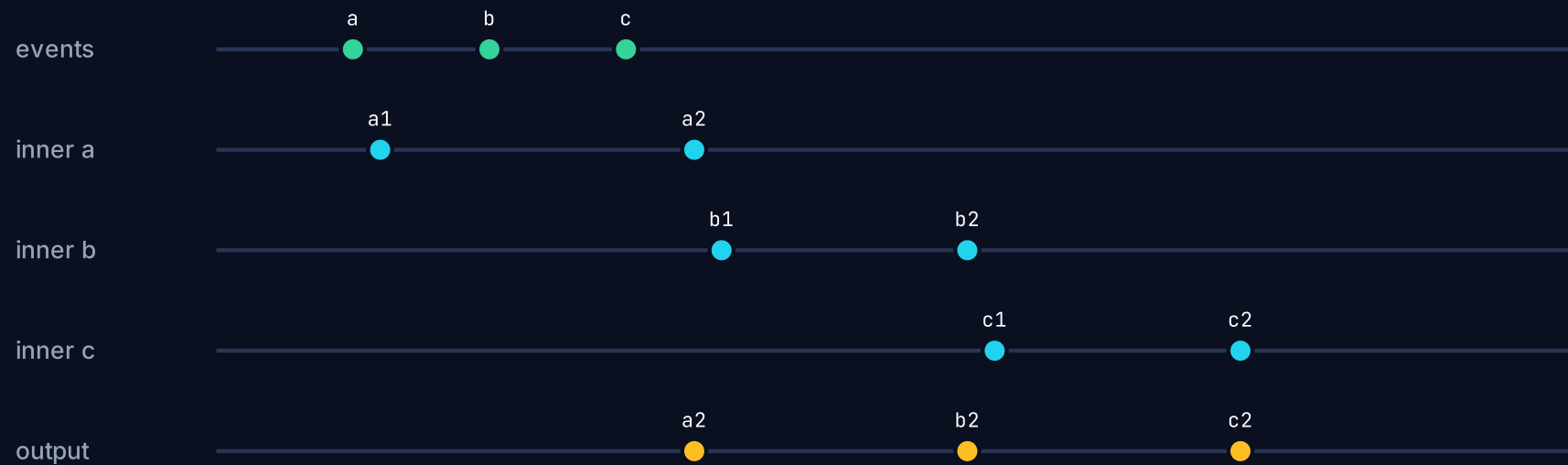
Поиск — Race condition: ответы могут прийти вразнобой

```
from([1, 2, 3]).pipe(  
  mergeMap(id => http.get(`/api/item/${id}`))  
  
).subscribe(item => log(item));
```

ANGULAR Загрузка нескольких независимых ресурсов параллельно.

⚡ mergeMap = «запусти всё, результат — когда готов».

concatMap — ставит в очередь



Внутренний поток ждёт завершения предыдущего

Порядок гарантирован: $a \rightarrow b \rightarrow c$

concatMap — когда использовать

Внутренние Observable выполняются строго по очереди: следующий ждёт завершения предыдущего.

ПОВЕДЕНИЕ

Новое событие становится в очередь.
Выполняется, когда предыдущее завершено.

ИДЕАЛЬНО ДЛЯ

Последовательные действия: сохранить →
загрузить → обновить

ОПАСНО ДЛЯ

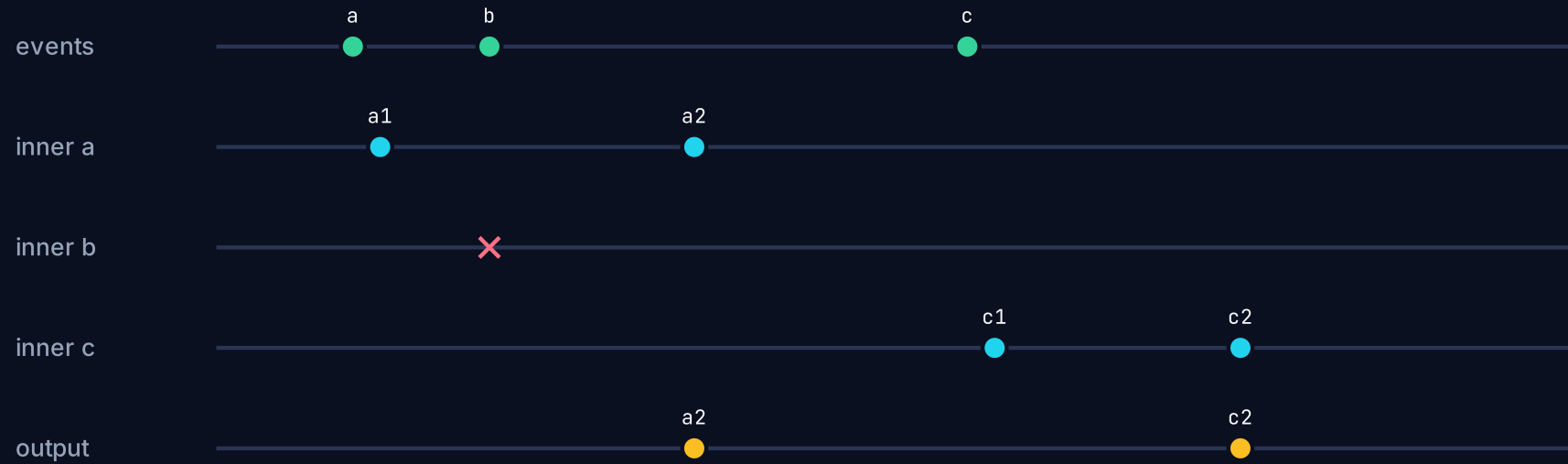
Поиск — каждый запрос ждёт предыдущего,
пользователь ждёт

```
saveActions$.pipe(  
  concatMap(action => http.post('/api/save', action))  
).subscribe(res => log(res));
```

ANGULAR Последовательное сохранение: сначала форма, потом связанные данные, потом лог.

 concatMap = «по очереди, не торопясь».

exhaustMap — игнорирует новые события



Пока внутренний поток активен — новые события игнорируются
Только после complete — следующий принимается

exhaustMap — когда использовать

Пока внутренний Observable выполняется, новые события просто отбрасываются.

ПОВЕДЕНИЕ

Новое событие во время выполнения → игнорируется. После complete — следующее принимается.

ИДЕАЛЬНО ДЛЯ

Submit формы — защита от двойной отправки

ОПАСНО ДЛЯ

Поиск — быстрые нажатия теряются без ответа

```
submitClicks$.pipe(  
  exhaustMap(() => http.post('/api/save', formData))  
).subscribe(res => log(res));
```

ANGULAR Кнопка отправки формы: пока запрос идёт, повторные клики игнорируются.

⚡ exhaustMap = «занят — не мешай».

Сравнение higher-order операторов

Четыре стратегии управления внутренними потоками

Оператор	Стратегия	Когда
switchMap	Отменяет предыдущий	Поиск, отмена устаревших
mergeMap	Параллельно все	Независимые задачи
concatMap	По очереди	Последовательные шаги
exhaustMap	Игнорирует новые	Защита от дублей

⚡ switchMap отменяет. mergeMap параллелит. concatMap ждёт. exhaustMap игнорирует.

Что теперь умеем

✓ Понять, что такое higher-order Observable

✓ Выбрать mergeMap для параллельных независимых задач

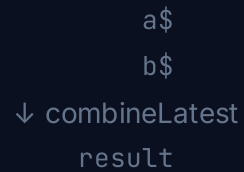
✓ Выбрать exhaustMap для защиты от двойных действий

✓ Выбрать switchMap для отмены устаревших запросов

✓ Выбрать concatMap для последовательных шагов

combineLatest()

Объединяет последние значения из нескольких потоков. Эмитит, когда каждый источник хотя бы раз отправил значение.



```
import { combineLatest } from 'rxjs';
```

```
combineLatest([
```

```
  this.filterForm.valueChanges,
```

```
  this.route.queryParams
```

```
]).subscribe(([filters, params]) =>  
  this.loadData(filters, params)  
);
```

ANGULAR Синхронизация фильтров формы и URL-параметров: при изменении любого — перезагрузка данных.

 combineLatest эмитит при любом изменении источника, если все уже отправили хотя бы раз.

forkJoin()

Ждёт завершения всех источников, затем эмитит последние значения. Как `Promise.all` для `Observable`.

```
httpA$  
httpB$  
↓ forkJoin  
result
```

```
import { forkJoin } from 'rxjs';
```

```
forkJoin({
```

```
  users: this.http.get('/api/users'),
```

```
  roles: this.http.get('/api/roles')
```

```
}).subscribe(({ users, roles }) =>
```

```
  this.init(users, roles)  
});
```

ANGULAR Загрузка страницы: параллельные HTTP-запросы, рендер только когда все готовы.

⚡ `forkJoin` ждёт `complete` всех источников. Если источник бесконечный — результата не будет.

takeUntil()

Завершает поток, когда управляющий Observable отправляет значение. Классический способ очистки подписок.



```
import { Subject, takeUntil } from 'rxjs';

private destroy$ = new Subject<void>();

ngOnInit() {

  this.api.getData().pipe(

    takeUntil(this.destroy$)

  ).subscribe(data => this.data = data);
}
```

ANGULAR Классический паттерн Angular: takeUntil(destroy\$) + ngOnDestroy для безопасной отписки.

⚡ takeUntil(notifier\$) завершает поток при первом значении notifier\$. В Angular 16+ используйте takeUntilDestroyed().

shareReplay()

Кэширует и переотправляет последние N значений новым подписчикам. Превращает cold Observable в warm.

Без shareReplay

Каждый subscribe → новый HTTP-запрос. N подписчиков = N запросов.

С shareReplay(1)

Первый subscribe → запрос. Остальные получают кэшированный результат.

Warm Observable

После первого подписчика — данные готовы для всех остальных мгновенно.

```
import { shareReplay } from 'rxjs';

@Injectable({ providedIn: 'root' })
export class ConfigService {

  private config$ = this.http.get('/api/config').pipe(

    shareReplay({ bufferSize: 1, refCount: true })
  );

  getConfig() { return this.config$; }
}
```

ANGULAR Конфигурация приложения: один HTTP-запрос, результат доступен всем компонентам.



shareReplay(1) — кэш последнего значения. refCount: true — отписка от источника, когда нет подписчиков.

Что теперь умеем

✓ Объединить потоки через `combineLatest` и `forkJoin`

✓ Завершить поток по сигналу через `takeUntil`

✓ Кэшировать результат через `shareReplay`

✓ Выбрать правильный оператор комбинирования



RxJS в Angular

HttpClient, Router, Forms, AsyncPipe, lifecycle

HttpClient

Возвращает **Observable** для каждого HTTP-запроса — один **next**, затем **complete** или **error**.

```
import { HttpClient } from '@angular/common/http';

@Injectable({ providedIn: 'root' })
export class UserService {
  constructor(private http: HttpClient) {}

  getUser(id: number) {
    return this.http.get<User>(`/api/users/${id}`);
  }
}
```

ANGULAR `this.userService.getUser(1).subscribe(user ⇒ ...)` — один ответ, затем **complete**.

 `HttpClient.get()` — это **Observable**. Если нужен повтор — переподпишитесь.

Router

`ActivatedRoute.params` — поток параметров URL. Каждая навигация — новое значение.

```
import { ActivatedRoute } from '@angular/router';

@Component({...})
export class UserDetailsComponent {
  constructor(route: ActivatedRoute) {

    route.paramMap.pipe(

      switchMap(params => this.api.get(params.get('id')))
    ).subscribe(user => this.user = user);
  }
}
```

ANGULAR При смене URL `/users/1` → `/users/2` — `switchMap` отменяет старый запрос, начинает новый.

 Router — источник событий. `params`, `queryParams`, `data` — всё Observable.

Reactive Forms

`valueChanges` и `statusChanges` — потоки значений и статусов формы.

```
export class SearchComponent {  
  search = new FormControl('');  
  
  results$ = this.search.valueChanges.pipe(  
    debounceTime(300),  
    distinctUntilChanged(),  
    switchMap(term => this.api.search(term))  
  );  
}
```

ANGULAR Поле поиска: `debounce` → фильтр дублей → отмена старого запроса → новые результаты.

⚡ `valueChanges` — это `Observable`. `pipe()` превращает его в готовый поток данных.

AsyncPipe и Templates

AsyncPipe подписывается и отписывается автоматически. Не нужно ручное управление.

component `users$ = this.api.getUsers();`



template `*ngIf="users$ | async as users"`



результат подписка создана при рендере, отписка при уничтожении

```
@Component({
  template: `
    <div *ngIf="users$ | async as users">
      @for (u of users; track u.id) {

        <app-user-card [user]="u" />
      }
    </div>
  `,
})
export class UserListComponent {
  users$ = this.api.getUsers();
}
```

ANGULAR AsyncPipe идеален для потоков данных в шаблоне: нет утечек, нет ручного unsubscribe.

⚡ AsyncPipe = subscribe при создании + unsubscribe при уничтожении. Автоматически.

Очистка подписок в современном Angular

`takeUntil(destroy$)` — классический подход. В Angular 16+ — `takeUntilDestroyed()`.

Классический

`takeUntil(destroy$) + OnDestroy`

Современный (Angular 16+)

`takeUntilDestroyed()` — без boilerplate

Декларативный

`AsyncPipe` — подписка и отписка автоматически

```
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';

@Component({...})
export class MyComponent {
  data$ = this.api.getData().pipe(
    takeUntilDestroyed()

  ); // отписка при уничтожении компонента
}
```

ANGULAR `takeUntilDestroyed()` — самый короткий путь к безопасной подписке в Angular 16+.

⚡ Если можно `AsyncPipe` — используйте `AsyncPipe`. Если нельзя — `takeUntilDestroyed()`.

Что теперь умеем

✓ Понять, что `HttpClient` возвращает `Observable`

✓ Использовать `valueChanges` для реактивных форм

✓ Безопасно очищать подписки через `takeUntilDestroyed()`

✓ Работать с `Router params` через `switchMap`

✓ Применять `AsyncPipe` для автоматической подписки

Паттерны решения задач

Ситуация → потоки → оператор → схема → код → типичная
ошибка

CHALLENGE

Live search без лишних запросов

СИТУАЦИЯ

Пользователь вводит текст в поле поиска. На каждый символ летит запрос — это лишняя нагрузка.

ПОТОКИ

valueChanges формы → поток строк (ввод пользователя)

ОПЕРАТОР

debounceTime + distinctUntilChanged + switchMap

```
input
  ↓ debounceTime(300) + distinctUntilChanged()
  filtered
    ↓ switchMap(term → HTTP)
    output
```

```
this.search.valueChanges.pipe(
  debounceTime(300),
  distinctUntilChanged(),
  switchMap(term => this.http.get('/api/search?q=' + term))
).subscribe(results => this.results = results);
```

ТИПИЧНАЯ ОШИБКА Отправлять запрос на каждый keystroke — сервер заваливается запросами, UI лагает.

CHALLENGE

Отмена устаревшего HTTP-запроса

СИТУАЦИЯ

Быстрая навигация между страницами: старый запрос ещё не вернулся, а нужен уже новый.

ПОТОКИ

Router params → поток ID → HTTP-запрос для каждого ID

ОПЕРАТОР

`switchMap`

```
params  
↓ switchMap(id → HTTP)  
inner
```

```
route.paramMap.pipe(  
  switchMap(params => this.api.get(params.get('id')))  
)  
).subscribe(user => this.user = user);
```

ТИПИЧНАЯ ОШИБКА Использовать `mergeMap` — результаты приходят вразнобой, пользователь видит данные от старого запроса.

CHALLENGE

Сохранение формы без двойного submit

СИТУАЦИЯ

Пользователь кликает «Сохранить» несколько раз. Каждый клик отправляет запрос.

ПОТОКИ

button click → поток событий → HTTP POST для каждого

ОПЕРАТОР

`exhaustMap`

`clicks`
↓ `exhaustMap(() → POST)`
`requests`

```
fromEvent(submitBtn, 'click').pipe(  
  exhaustMap(() => this.http.post('/api/save', this.form.value))  
)  
.subscribe(res => this.onSaved(res));
```

ТИПИЧНАЯ ОШИБКА Использовать `mergeMap` — каждый клик отправляет новый POST, данные сохраняются дважды.

Загрузка данных из нескольких источников

`combineLatest` — объединяет последние значения из нескольких потоков. `forkJoin` — ждёт завершения всех.

```
users$
roles$
↓ combineLatest
combined -
```

```
import { combineLatest } from 'rxjs';
import { forkJoin } from 'rxjs';
```

```
// Когда оба потока активны и обновляются:
```

```
combineLatest([users$, roles$])
  .subscribe(([users, roles]) => render(users, roles));
```

```
// Когда нужно дождаться завершения всех:
```

```
forkJoin([http.get('/a'), http.get('/b')])
  .subscribe([a, b] => init(a, b));
```

ANGULAR Загрузка страницы: параллельные HTTP-запросы, рендер только когда все готовы.

⚡ `combineLatest` — для живых потоков. `forkJoin` — для завершающихся (HTTP).

Обработка ошибок без поломки UI

`catchError` — перехват ошибки и возврат альтернативного потока. Поток не умирает.

```
http$  
↓ catchError(err → of([]))  
safe$
```

```
this.http.get<User[]>('/api/users').pipe(  
  catchError(() => of([])),      // fallback на пустой массив  
  
  startWith([])                 // начальное состояние  
  
).subscribe(users => this.users = users);
```

ANGULAR HTTP-запрос упал — показать пустой список вместо красного экрана ошибки.

⚡ `catchError` должен вернуть `Observable`. `of([])` — безопасный `fallback`.

CHALLENGE

Предотвращение утечек подписок

СИТУАЦИЯ

Компонент уничтожен, но подписка живёт.
Коллбэки вызываются на уничтоженном компоненте.

ПОТОКИ

Любой long-lived Observable: interval, valueChanges, Subject

ОПЕРАТОР

`takeUntilDestroyed()` или `AsyncPipe`

без очистки `subscribe` → `component destroyed` → `callback` всё ещё вызывается!
с `takeUntilDestroyed` `subscribe` → `component destroyed` → `unsubscribe` автоматически

```
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
```

```
// В поле компонента:
```

```
data$ = this.api.getData().pipe(
```

```
  takeUntilDestroyed()
```

```
);
```

```
// Или в конструкторе:
```

```
this.api.getLive().pipe(
```

```
  takeUntilDestroyed()
```

```
).subscribe(v => this.v = v);
```

ТИПИЧНАЯ ОШИБКА Сохранять Subscription в свойстве и отписываться в `ngOnDestroy` — работает, но boilerplate. Легко забыть.

CHALLENGE

Синхронизация фильтров, URL и API

СИТУАЦИЯ

Фильтры формы, параметры URL и API-запрос должны быть синхронизированы. Изменение любого — перезагрузка данных.

ПОТОКИ

valueChanges формы + route.queryParams → два потока

ОПЕРАТОР

`combineLatest` + `switchMap`

```
filters$
params$
↓ combineLatest + switchMap([[f, p]] → API)
output
```

```
combineLatest([
  this.filterForm.valueChanges.pipe(startWith(this.filterForm.value)),
  this.route.queryParams
]).pipe(
  switchMap([[filters, params]] =>
    this.http.get('/api/data', { params: { ...filters, ...params } })
  )
).subscribe(data => this.data = data);
```

ТИПИЧНАЯ ОШИБКА Подписываться на каждый источник отдельно — race condition, данные рассинхронизируются.

CHALLENGE

Состояния loading / data / error

СИТУАЦИЯ

HTTP-запрос должен показывать loading, данные или ошибку. При ошибке — возможность повтора.

ПОТОКИ

HTTP-запрос → состояния loading, data, error

ОПЕРАТОР

`startWith + catchError + retry`

```
http$  
↓ startWith('loading') + catchError + retry(2)  
view$  
error
```

```
this.http.get<Data>('/api/data').pipe(  
  retry({ count: 2, delay: 1000 }),  
  
  catchError(err => of({ error: err })),  
  
  startWith({ loading: true }),  
  
  finalize(() => this.loading = false)  
)  
.subscribe(state => this.state = state);
```

ТИПИЧНАЯ ОШИБКА Не обрабатывать error — поток умирает, UI застревает в loading. catchError без fallback — тоже умирает.



ИТОГИ

Cheat sheet и алгоритм выбора оператора

Cheat sheet: ситуация → оператор

Краткий справочник для быстрого выбора

Ситуация	Оператор	Почему
Ждать паузы ввода	<code>debounceTime</code>	Не слать запрос на каждый символ
Убрать дубли подряд	<code>distinctUntilChanged</code>	Не повторять одно и то же
Отменить устаревший запрос	<code>switchMap</code>	Новый запрос → старый не нужен
Защита от двойного submit	<code>exhaustMap</code>	Пока занят — игнорируй клики
Последовательные шаги	<code>concatMap</code>	Очередь: шаг за шагом
Параллельные запросы	<code>mergeMap</code>	Все сразу, результаты по готовности
Ждать все запросы	<code>forkJoin</code>	Как Promise.all для Observable
Объединить живые потоки	<code>combineLatest</code>	Последние значения из каждого
Начальное состояние	<code>startWith</code>	Показать 'Loading' до данных
Перехват ошибки	<code>catchError</code>	Вернуть fallback, не уронить UI
Очистка подписки	<code>takeUntilDestroyed</code>	Отписка при уничтожении
Побочный эффект	<code>tap</code>	Лог, отладка — без изменения потока

 Сохраните этот cheat sheet — он покрывает 90% задач из Angular Challenges.

Алгоритм выбора higher-order оператора

Четыре вопроса — и вы знаете, какой оператор нужен

1 Нужно отменять прошлую работу при новом событии?

→ **switchMap**

Live search, навигация, отмена устаревших запросов

2 Нужна строгая очередь — шаг за шагом?

→ **concatMap**

Последовательные сохранения, цепочки запросов

3 Можно выполнять параллельно, порядок не важен?

→ **mergeMap**

Логирование, независимые задачи, фоновые процессы

4 Нужно игнорировать повторные действия, пока текущее не закончилось?

→ **exhaustMap**

Submit формы, предотвращение двойной отправки

⚡ switchMap отменяет. concatMap ждёт. mergeMap параллелит. exhaustMap игнорирует.

RxJS

Поток значений во времени — одна модель для всех источников

● Observable ● Operators ● Events ● Results ● Errors

Источники:

rxjs.dev · rxjs-course-avy.web.app · angular-challenges.vercel.app